Stocktrails

FINTECHATHON 2026 · INTERNATIONAL TRACK

Technical documentation

System architecture, core algorithms and security design of a behavioural finance agent that learns from what an investor researches and what they then trade.

THEME / TOPIC	RELEASE DESCRIBED	DOCUMENT	EVIDENCE
AI as a Financial Participant · Topic A, Personal Finance Assistant (parts of B)	Extension 0.4.0, web app and Supabase backend · build `c711d16`	Version 1.0 · 19 October 2026	423 automated checks, 0 failing · synthetic data only

---

Summary

1 System architecture

- 1.1 Components and trust zones
- 1.2 Where each kind of data lives
- 1.3 Capture and sync
- 1.4 Technology and deployment

2 Core algorithms

- 2.1 Pipeline
- 2.2 Decisions and outcomes
- 2.3 The circular shift test
- 2.4 Rules and walk-forward validation
- 2.5 Adherence and the pre-trade check
- 2.6 The assistant agent loop
- 2.7 Two agents with paper portfolios
- 2.8 Calibration results

3 Security design

- 3.1 Data minimisation
- 3.2 Permission tiers
- 3.3 Cloud authorisation
- 3.4 Controls
- 3.5 Security review and known risks
- 3.6 Compliance

4 Tests and reproduction

5 Limits

Summary

Stocktrails is a browser extension, installable web app and mobile app. It records which stocks an investor looks at on finance portals, when, for how long and on which kind of page, together with the calls they make (Positive, Neutral or Negative, with an optional target, horizon and note). Investors can import their broker's transaction export, which is read and stored only on their device.

On top of these two records sits a behavioural finance agent. It **recognises** the investor's own biases with statistical tests, **recommends a concrete rule** for each only after checking that the rule would have improved later trades it had not seen, lets the investor **adopt** the rule with one click, then **tracks adherence** and checks intended buys against adopted rules. An assistant answers questions about the investor's data by choosing read-only tools that run on the device.

Statistics decide

Every verdict comes from deterministic, tested code. The language model chooses tools and words the result.

Trades stay local

Imported trades, amounts and identifiers never leave the device. Three filters guard what does.

Out of sample first

A rule is recommended only if it held up on later decisions. In 24 worlds with no effect, 0 of 216 rules passed.

No order path

No tool, endpoint or credential exists that could place a trade. Every change is a click.

1 System architecture

The product is local-first. Three clients share one set of JavaScript modules: a Chrome MV3 extension (the primary client), a progressive web app that runs the same dashboard with `chrome.*` shimmed over IndexedDB, and a React Native app for Android and iOS. A Supabase project in the EU provides sign-in, synchronisation of attention data between a user's devices, anonymous community aggregates and a small set of edge functions. The heavy analysis runs in the browser.

1.1 Components and trust zones

Figure 1 groups the components by where they run. The device zone holds all raw data and all analytics. The cloud zone holds only attention events that the user chose to sync and per-user counters. Third-party services receive price requests, server-side data collection requests, or de-identified results for the model.

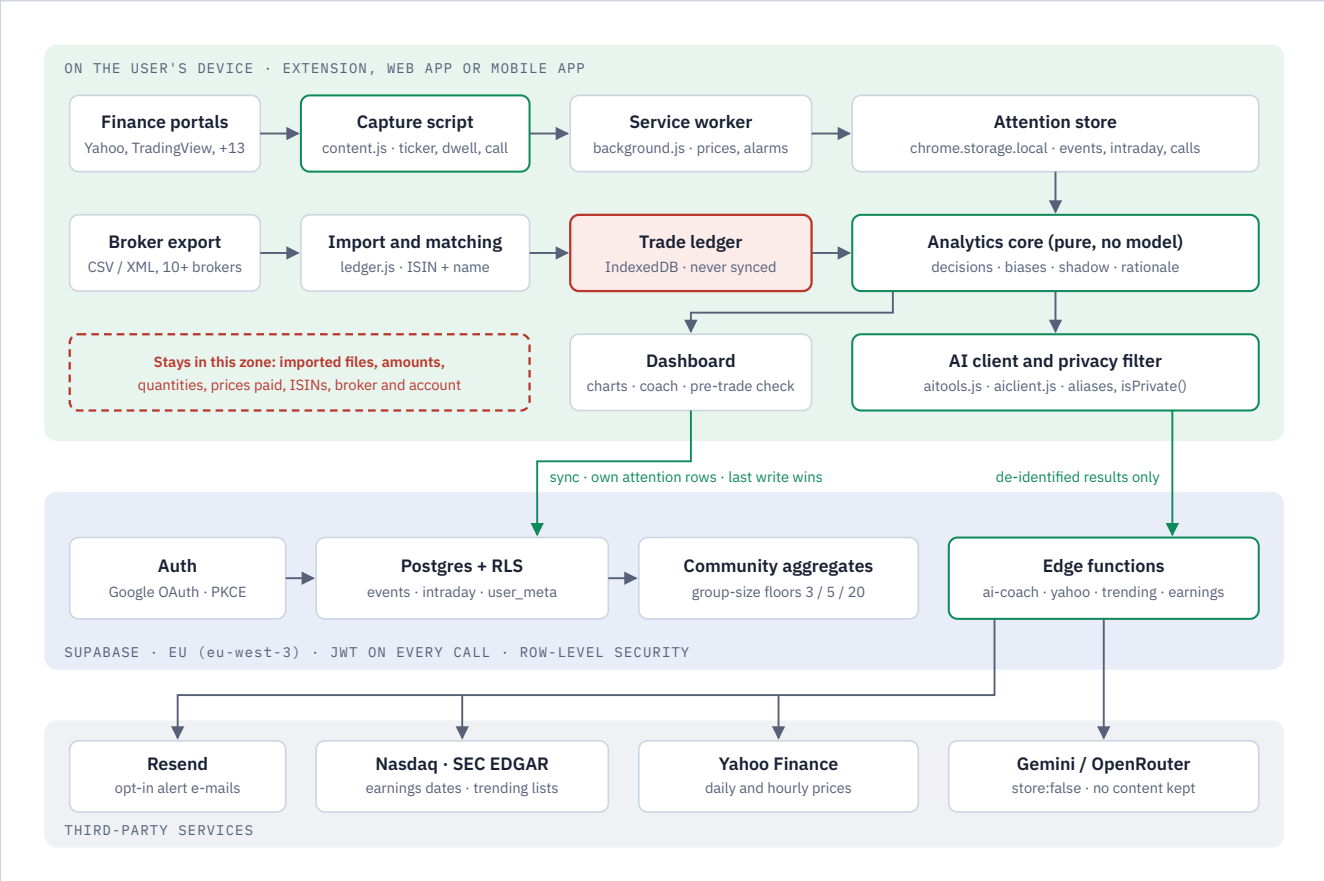


Figure 1. Components and trust zones. Green arrows are the only two paths by which user data leaves the device. The extension also fetches prices from Yahoo directly; the web app uses the `yahoo` proxy function because Yahoo sends no CORS headers.

COMPONENT	CODE	ROLE
Capture script	`src/content.js`, `src/portals.js`	Detects the ticker from the URL (15 built-in portals and user-defined patterns), falls back to page content (cashtags, exchange-qualified symbols, tracked names). Measures active visible time. Renders the call card in a closed shadow root.
Service worker	`src/background.js`	Message router, price at the moment of the view, hourly price archive, alarms (sync every 5 min, daily prices), viewing sessions for the AI agent's live verdict.
Local stores	`store.js`, `ledgerdb.js`, `pricedb.js`, `pricecache.js`	Attention events and settings in `chrome.storage.local`; trade ledger, adopted rules, agent decisions and pre-trade checks in IndexedDB; price archives in IndexedDB.
Analytics core	`decisions.js`, `biases.js`, `shadow.js`, `rationale.js`, `indicators.js`	Pure functions, no I/O and no model: decisions and outcomes, bias tests, rule fitting and walk-forward validation, adherence, paper portfolios, statistics on notes.
AI client	`aitools.js`, `aiclient.js`	Read-only tool catalogue built from allowlists, aliases, privacy check, the agent loop, number checking, action cards and the request log.
Cloud	`supabase/`	Postgres with row-level security, Google sign-in, community RPCs with group-size floors, 7 edge functions (`ai-coach`, `yahoo`, `trending`, `earnings`, `crowd-alerts`, `geo`, `feedback-dispatch`).

1.2 Where each kind of data lives

DATA	STORED	LEAVES THE DEVICE	DELETED BY
Attention events (ticker, time, portal, page section, dwell, price at view, call)	Device; Supabase after sign-in	`own account only`	Delete all, or account deletion
Imported broker trades, matches	IndexedDB `stocktrails_ledger`, per account	`never`	Delete all; not in JSON backup

DATA	STORED	LEAVES THE DEVICE	DELETED BY
Adopted rules, reminders, review notes	IndexedDB	`never`	Delete all
Paper decisions of the agents, pre-trade checks	IndexedDB stores `agent`, `checks`	`never`	Delete all
Tool results and questions sent to the assistant	Local request log (last 80 entries)	`de-identified, on consent`	Clear log; sign-out
Assistant usage	Supabase `ai_usage`: user id, day, count	counter only	Account deletion
Community aggregates	Computed at query time from synced events	only above group-size floors	n/a

1.3 Capture and sync

Capture needs no click. The content script polls the URL every 1.5 seconds; a new ticker becomes an attention event. Each event is written twice: to the *daily* set, where repeat views within 60 minutes collapse into one row with a view counter, and to the *intraday* set, which keeps every view. All writes land locally first, so the dashboard works offline.

Synchronisation uses two cursors because push and pull ask different questions. The push cursor compares local timestamps with local timestamps. The pull cursor uses the server's own receive time (`synced_at`), because a row captured early but uploaded late would otherwise be invisible to devices whose cursor had moved past its capture time. Pulls page by keyset in blocks of 1000 rows, since PostgREST truncates responses at that size and offset paging skipped rows that changed during a pull.

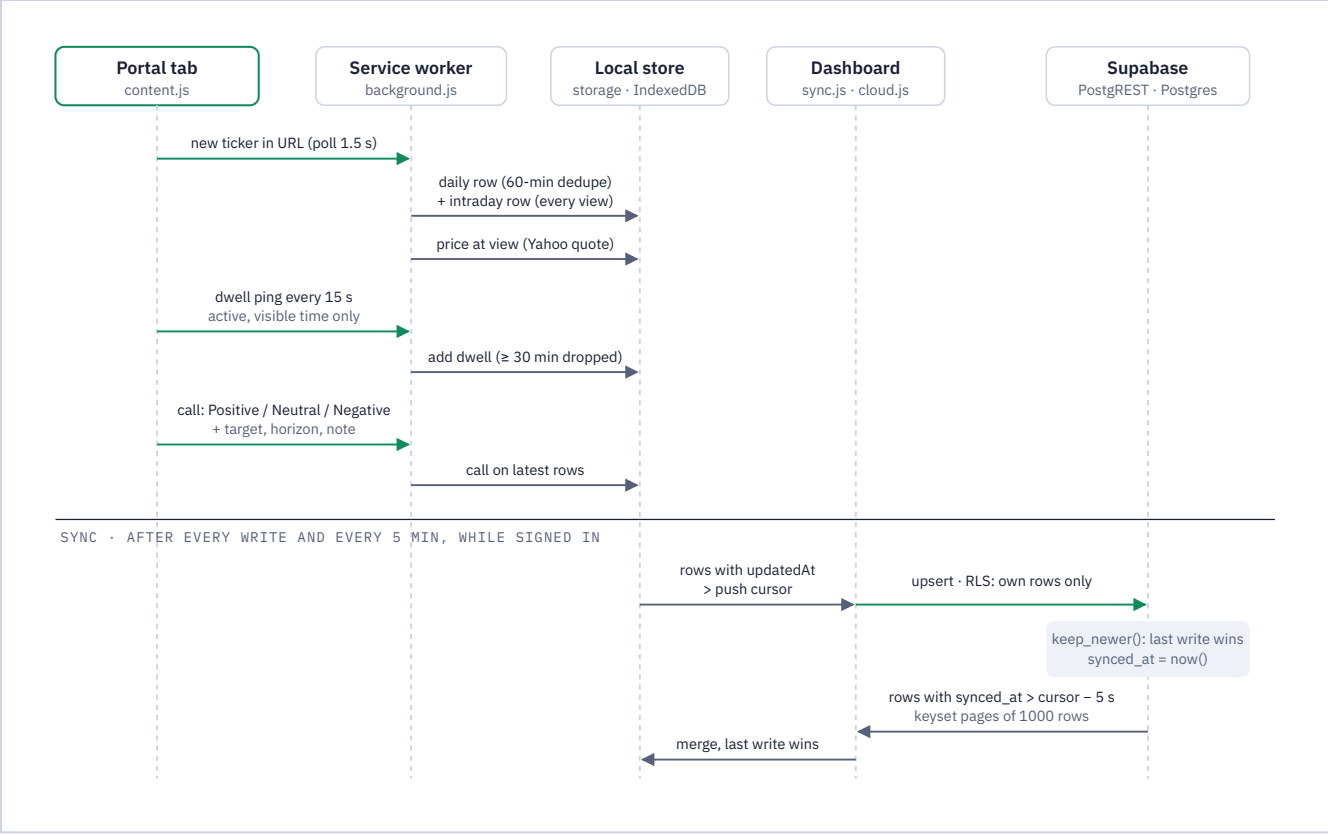


Figure 2. Capture and sync for one page view. A full reconcile that ignores both cursors runs once a day and whenever the sync protocol version changes, so any gap closes by itself. The trade ledger is not part of sync.

1.4 Technology and deployment

LAYER	TECHNOLOGY	DELIVERY
Extension	Chrome Manifest V3, plain JavaScript, no remote code, no build step	Chrome Web Store via GitHub Actions; Firefox and Safari packages
Web app	Same dashboard, `pwa-shim.js` over IndexedDB, service worker, share target	GitHub Pages; only web-app files are published
Mobile	React Native + Expo (TypeScript), Skia, ML Kit OCR	EAS build; reuses the Supabase backend unchanged
Backend	Supabase: Postgres 15, Auth, RLS, pg_cron, Vault, Deno edge functions	SQL migrations in `supabase/migrations/`; functions deployed with JWT verification on
Model	Google Gemini Interactions API (`store:false`); Cerebras or OpenRouter as alternatives; `AI_PROVIDER=mock` for tests	Called only from the `ai-coach` function; one module builds and reads requests, so the provider is replaceable

Deploying the AI layer takes three steps: apply `20261010_ai_usage.sql`; set `GEMINI_API_KEY` and `AI_ALLOWED_USERS` (empty means nobody); deploy `ai-coach`. The judging sandbox needs none of this.

2 Core algorithms

The central design decision is the division of labour. Everything that produces a number or a verdict is deterministic JavaScript, tested without a model. The model is used for four things only: choosing which read-only tools to run for a question, putting the returned results into words, proposing actions the user may approve, and, as one of two paper-trading agents, making decisions that are compared with the user's and the rules agent's.

2.1 Pipeline

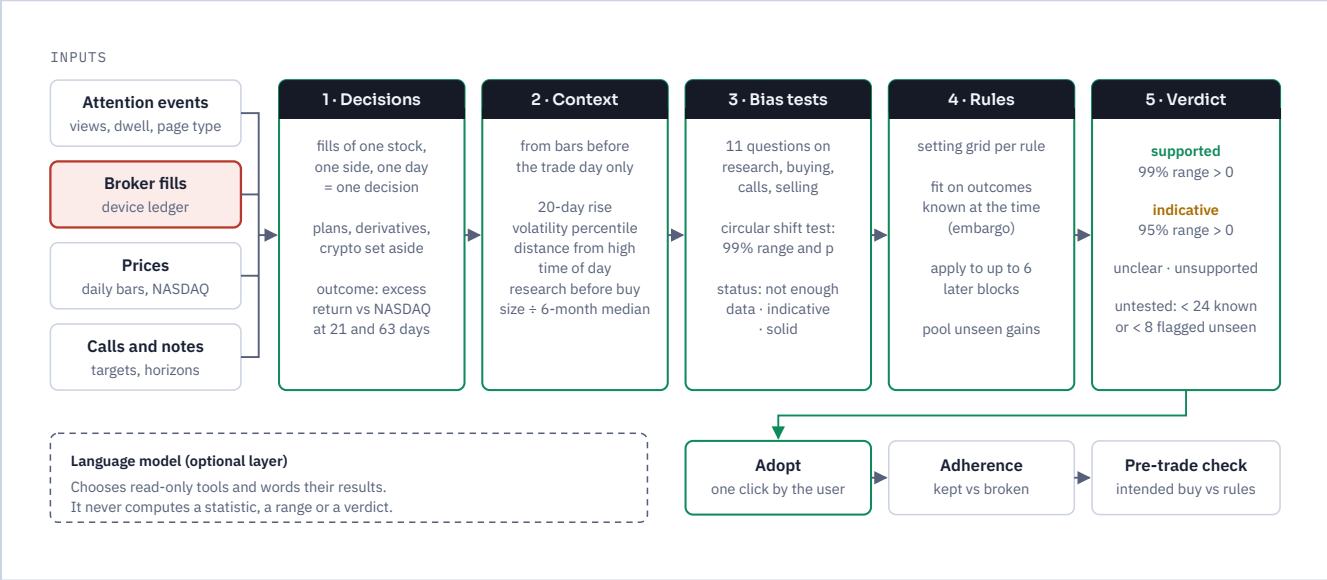


Figure 3. From raw records to a recommendation. Steps 1 to 5 are pure functions in `src/decisions.js` and `src/biases.js`. A rule reaches the user only from step 5, and changes behaviour only after the user adopts it.

2.2 Decisions and outcomes

Fills of one instrument on one side on one day form a **decision**. Savings-plan executions, derivatives, bonds and crypto are set aside, as are decisions made before any research was recorded. The outcome of a buy is its excess return over the NASDAQ Composite (`^IXIC`) from the close of the trade day, over 21 trading days, and over 63 days for calls. Holdings that existed before the export begins are inferred from sales larger than the known position.

Context is measured from bars *before* the trade day, so nothing the investor could not have seen enters a feature. Volatility is the standard deviation of 20 daily returns; the buy is then placed by percentile among all trading days of the covered period, so that "you buy in turbulent markets" is judged against how often markets are turbulent. Size is judged against the investor's median buy over the previous 6 months, so a growing portfolio does not read as buying bigger.

2.3 The circular shift test

Markets stay calm or turbulent for weeks, and buying comes in bursts. Neighbouring days and neighbouring buys are therefore alike, and an ordinary bootstrap gives ranges that are too narrow. Instead, the whole pattern of buys is slid along the calendar by every possible offset, wrapping around at the end, and each statistic is recomputed. The buys keep their spacing and the market keeps its stretches; only their alignment is broken. The distribution over all offsets gives the 99% range and the p-value of the observed statistic (`shiftTest`).

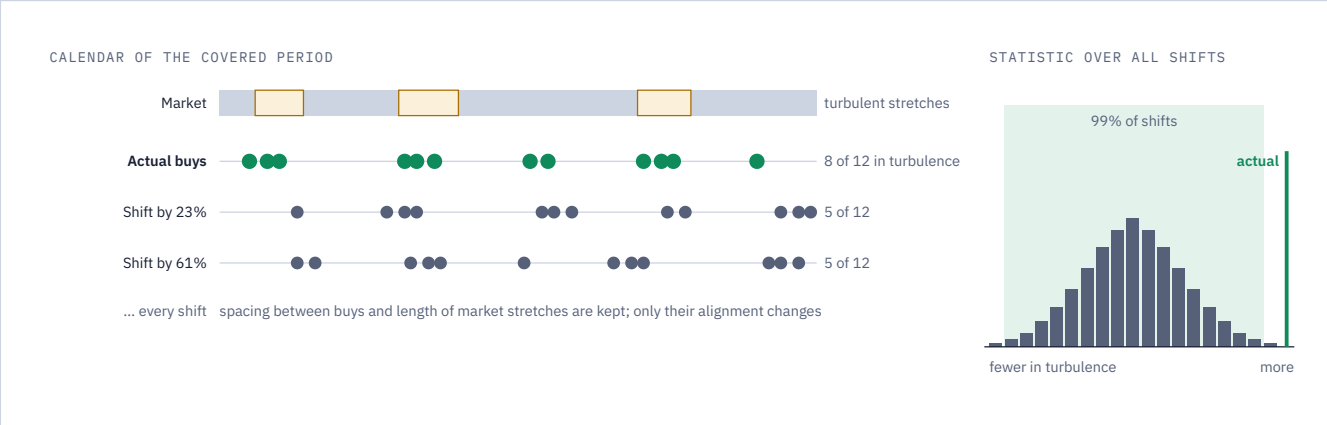


Figure 4. The shift test on an illustrative pattern of 12 buys. If the actual share of buys in turbulent markets lies outside the range produced by all shifts, the alignment of buying with turbulence is unlikely to be chance.

2.4 Rules and walk-forward validation

Each bias maps to one or more rules, for example "wait after a run-up", "sleep on it", "avoid your worst time of day" or "cap the size in turbulent markets". A rule has a grid of settings, a *flag* (the rule says do not) and a *gain*: what not doing it would have been worth against the NASDAQ, assuming the skipped money earns the market. A cap rule scales the size instead of skipping.

`fit` chooses the setting that maximises

```
score(s) = mean_gain(s) × √ n_flagged(s)

subject to  n_flagged(s) ≥ 5,  n_flagged(s) ≤ 0.6 × n,  mean_gain(s) > 0
```

`walkForward` orders decisions in time. The first half is only ever used for training. The rest is cut into up to six blocks. For each block, the rule is fitted on decisions whose outcome was **already known when the block began**: a buy's outcome takes a month to arrive, so the decisions of the month before the block are embargoed. The fitted setting is then applied to the block. A fold in which no setting helped recommends nothing for that block. Only the gains on the later blocks are pooled and given a bootstrap range.

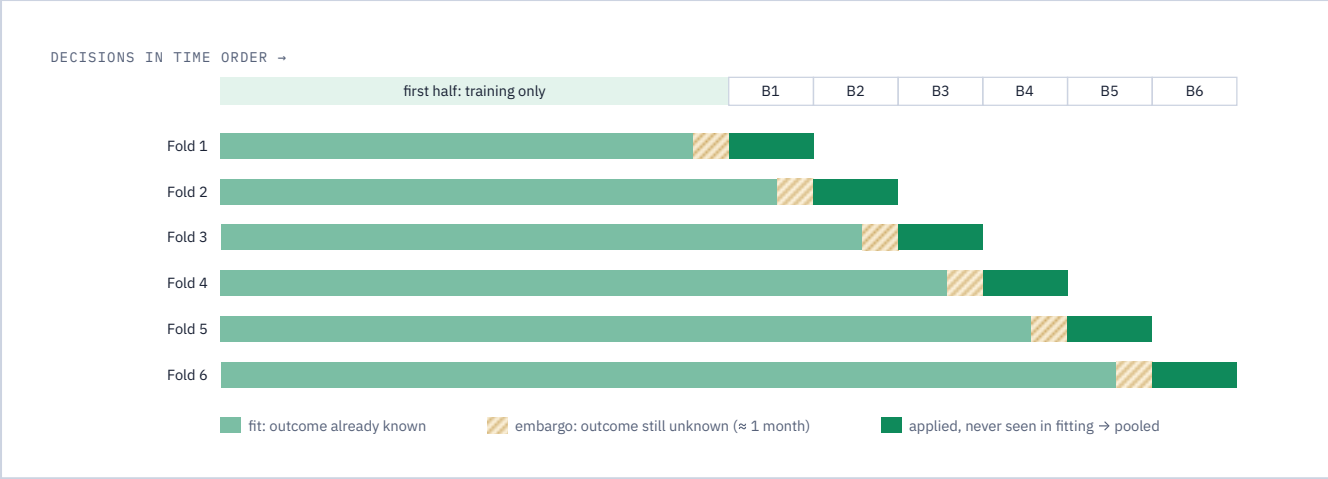


Figure 5. Chronological walk-forward. Each block is judged only by a rule fitted on decisions that had finished by the time the block began. The pooled gains from the six test blocks give the verdict in Table 1.

VERDICT	CONDITION	WHAT THE USER SEES
supported	99% bootstrap range above zero	Recommended, with setting, gains and ranges
indicative	95% range above zero	Shown as a lead, not recommended
unclear	range includes zero	Not recommended
unsupported	range below zero, or in-sample only	Not recommended; the in-sample flattery is shown
untested	fewer than 24 decisions with a known outcome, or fewer than 8 flagged out of sample	"Not enough data", a result in itself

Table 1. Verdict on the pooled out-of-sample gain per flagged decision.

2.5 Adherence and the pre-trade check

After adoption, the coach reports the share of decisions that broke the rule against the same share over an equally long stretch before adoption, with a Newcombe interval for the change, and compares the outcome of decisions that kept the rule with those that broke it, with a bootstrap range. It is reported as an association, since nobody was assigned to comply. The pre-trade check takes an intended buy (stock and amount), measures the same features as of now, and lists which adopted rules object, which cannot be checked, and any reminder the user set for that stock. Every check is saved on the device with its inputs and the rule versions in force, so checks can be evaluated later.

2.6 The assistant agent loop

The assistant works in steps, at most five requests per question. Each request carries the question, with stocks replaced by aliases, and the results gathered so far. The model either names up to four more tools from a fixed read-only catalogue, or answers. The tools run in the browser, so the model can look further when a first result raises a question, for example by reading a rule's details after seeing the biases, or trying a rule with another setting. The fifth step must answer, and a tool call it already has is dropped. The browser then checks every number in the reply against the results and marks any it cannot find.

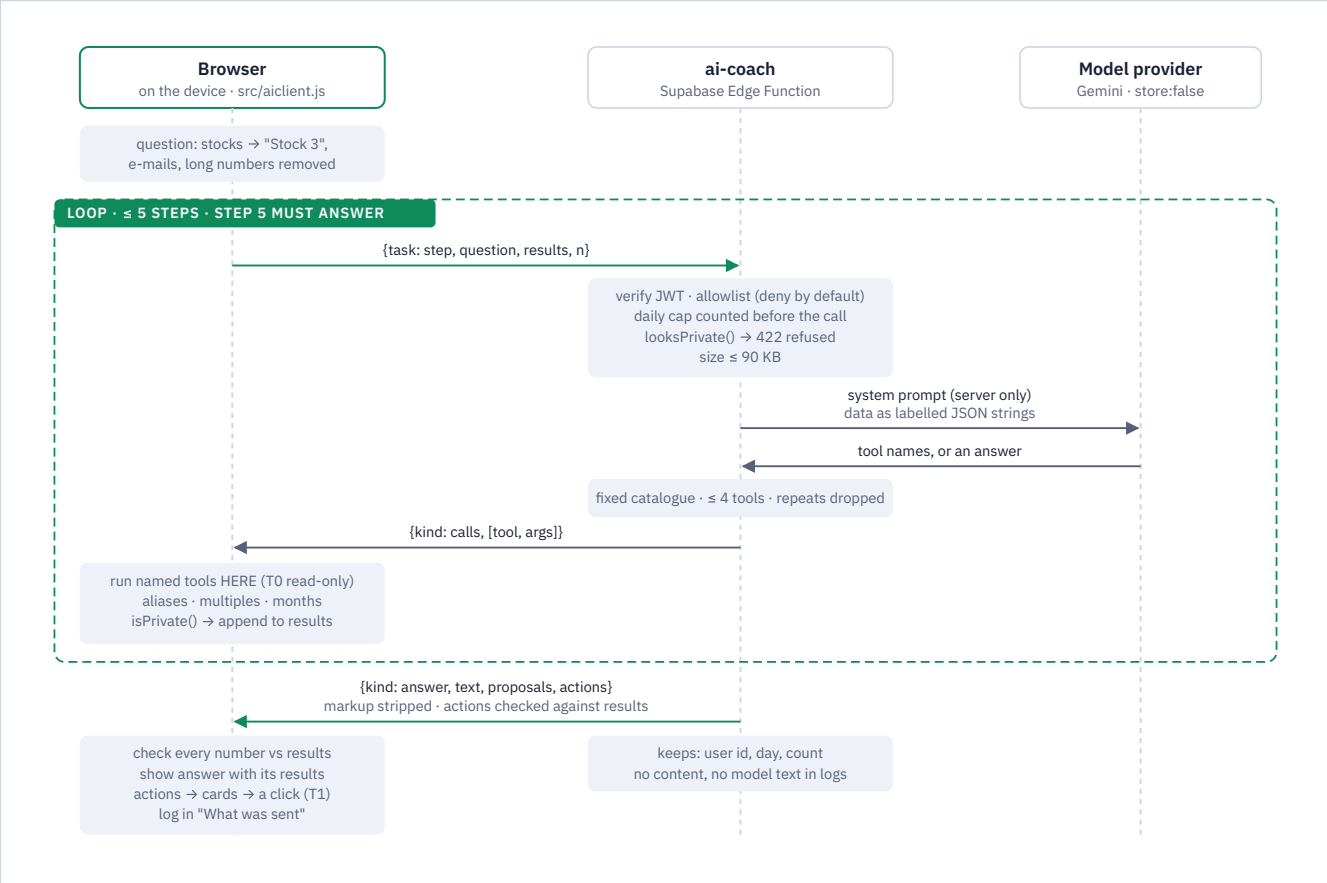


Figure 6. One question through the assistant. Raw data never crosses the dashed lifeline of the device; only tool results that passed the privacy check do. Every step counts against the user's daily cap.

TOOL (T0, READ-ONLY)	RETURNS
`get_overview`, `get_finding(q1…q11)`	Counts, shares, returns against the NASDAQ and statuses, as plain facts
`get_biases`, `get_rule(id)`, `simulate_rule(id)`	Biases, rule wording, the out-of-sample verdict
`list_decisions`	Alias, month, size as a multiple of the usual buy, research, rise, time of day, market, result; no amounts
`get_adherence`, `check_buy(stock, size)`	How rules are kept; an intended buy against adopted rules and reminders

TOOL (T0, READ-ONLY)	RETURNS
`compare_periods(days)`, `what_if(id, …)`	Whether a habit is fading; a rule with another setting over the whole history and its later half
`explain_decision`, `stock_summary`, `get_weekly_review`	One decision with its research and the rules that flag it; one stock's views, calls and decisions; the weekly review
`get_agent_portfolio`, `get_agent_decisions`	The paper portfolios as ratios; what each agent decided at single moments and why

2.7 Two agents with paper portfolios

To show what following the rules would have meant in money terms, two agents trade paper portfolios on the investor's own buys and on the stocks they looked at. The **rules agent** (`src/shadow.js`, no model) keeps, each month, only the rules that had held up out of sample on outcomes known by then. For each candidate it buys at once, waits until no active rule objects (re-checking 1 to 10 trading days later), or passes. The **AI agent** receives each candidate as of that day (alias, "day N", ratios, the rules agent's view) in requests of 15, and is asked only about moments at most two days old. Every decision is saved once with its time; there is no backfill, so the model cannot know what happened next.

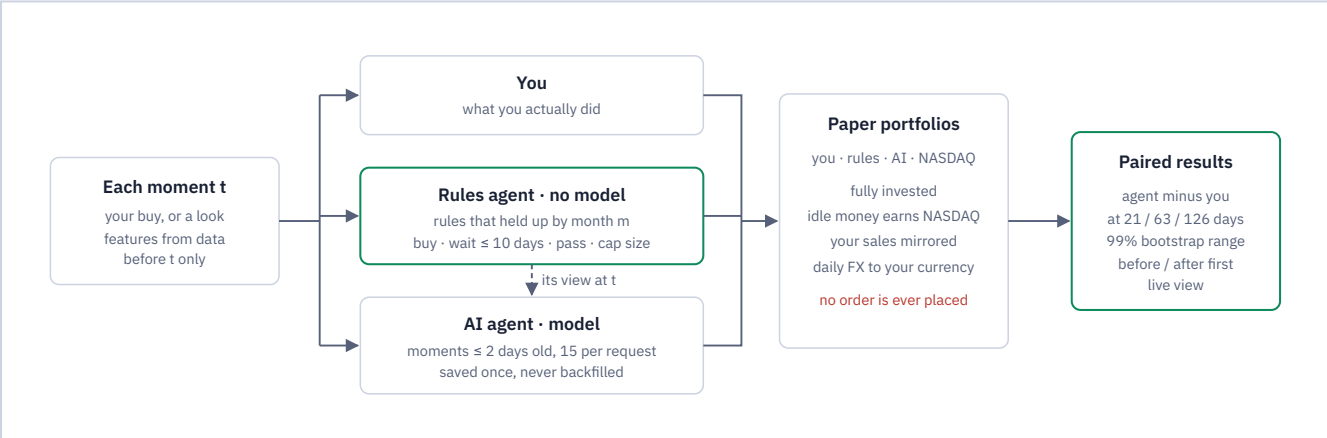


Figure 7. The agent simulation. Both agents act only on the investor's own buys and looks and only move paper portfolios drawn in the browser. Results are measured to the same end bar for each pair.

2.8 Calibration results

`tools/synth-world.js` builds deterministic synthetic investors with planted behaviours. The coach must find what is planted, recommend only what holds up on later trades, and find nothing where nothing exists. Table 2 is taken from the execution evidence (`docs/hackathon/evidence/demo-run.md`, seed 21, 140 analysed buys over about 31 months).

RULE	VERDICT	FLAGGED (LATER)	OUT OF SAMPLE	99% RANGE	IN SAMPLE
Wait after a run-up	supported	15 / 70	+16.3	+10.8 to +21.5	+14.2
Avoid your worst time of day	supported	19 / 70	+17.0	+12.3 to +21.1	+15.0
Sleep on it (impulse buys)	supported	27 / 62	+13.2	+9.2 to +17.2	+12.1
Research before you buy	supported	41 / 70	+11.5	+7.9 to +15.1	+10.0
Act on your calls or drop them	supported	53 / 89	+11.3	+6.5 to +16.0	+10.9
Pause in turbulent markets	supported	26 / 70	+10.9	+6.3 to +15.1	+10.9
Cap the size in turbulent markets	supported	18 / 68	+4.4	+2.6 to +6.2	+3.9

Table 2. Rules on a synthetic investor. Gains are percentage points against the NASDAQ per flagged decision, out of sample, with 99% ranges.

> **Null worlds.** In 24 synthetic worlds with no planted effect, 0 of 216 rules were called supported. An effect that stops halfway through the history is not recommended, and the embargo has its own unit tests.

3 Security design

The security design follows from one principle: the most sensitive data, an investor's trades, should not exist anywhere it does not need to. Raw trades are parsed in the browser and kept in IndexedDB on the device. The cloud receives attention events for the user's own account under row-level security, and the model receives only de-identified results that pass three filters. The agent has no path to an order.

Trust boundaries, in order of exposure: (1) finance portal pages, which are untrusted and may read the DOM around the capture card; (2) the device, which holds everything; (3) the Supabase project, which holds synced attention and counters; (4) model and data providers, which see only what the edge functions send.

3.1 Data minimisation

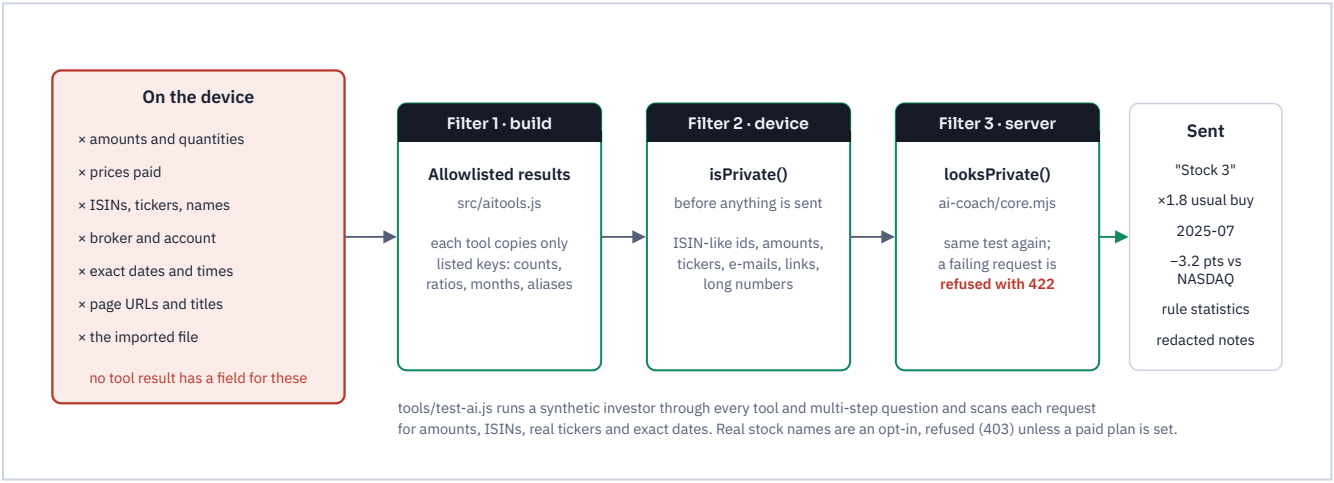


Figure 8. Defence in depth for everything sent to the model. Filter 1 makes leakage structurally hard; filters 2 and 3 catch what a future code change might let through.

3.2 Permission tiers

The agent's authority is fixed in code. The server validates every tool call and proposed action, and the browser executes only what the user confirms.

	What the agent may do	How it is enforced
T0 READ-ONLY	Name tools from a fixed catalogue that read results computed on the device. As the AI agent: say buy, wait 1 to 10 trading days or pass on a paper portfolio.	Server accepts only `TOOLS` in `core.mjs`, checks every argument, ≤ 4 tools per step, ≤ 5 steps. Paper decisions: asked ids only, three actions, clamped days, ≤ 15 per request.
T1 USER DECIDES	Propose an action: adopt, retire or retune a rule, set a pre-trade reminder for a stock, save a review note.	Validated against `ACTIONS`: known types, rule ids and stocks that appear in the results, at most 3. Shown as a card with what-if numbers; written locally only on a click, logged as "You approved".
T2 NOT OFFERED	**What the agent can never do** Place or change orders, hold broker or bank credentials, write trade data to any server, change account settings.	**How it is enforced** No tool, endpoint, scope or code path exists. The function has no database access beyond upserting a usage counter.

Figure 9. Permission tiers as implemented, with the test that covers each in `tools/test-ai.js` and `tools/test-shadow.js`.

3.3 Cloud authorisation

Row-level security is enabled on every table. Owner policies use `user_id = auth.uid()` for reading and writing. Admin reads go through `is_admin()`, which a user cannot set because `profiles` has a column-level UPDATE grant. Service-only tables (`ai_usage`, `crowd_notified`) have no policies, and their functions are revoked from `anon` and `authenticated`. Security-definer functions pin `search_path`. Community views return a cell only when enough distinct users stand behind it.

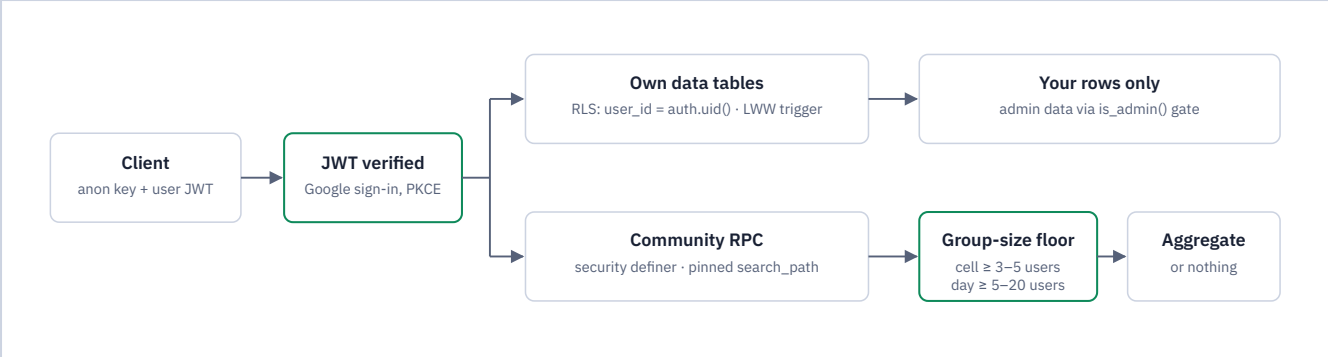


Figure 10. Authorisation path for a request. Cross-user data is only ever an aggregate, and only above a minimum group size (migration `20261017_community_privacy_floors.sql`).

3.4 Controls

AREA	CONTROL
Consent	The assistant is off by default. Explicit consent with a list of what is and is not sent; withdrawn in one click. Notes are previewed, redacted, before they are sent. Accepting the Terms and Privacy Policy is a separate, required step before sign-in.
Access	Deny by default: `AI_ALLOWED_USERS` lists who may use the function, empty means nobody. JWT verified. E-mail allowlist entries match Google sign-ins only.
Abuse and cost	Per-user daily cap counted in the database before the model is called; a failing counter stops the call. Every agent step counts. 90 KB request limit. Notes cached by hash and sent once.
Prompt injection	System prompts live only on the server. Question and results travel inside JSON strings labelled as data. Tool names are validated against the catalogue; no tool can write. Replies are validated, markup is stripped and the page renders text only.
Hallucination	Answers are built from results; every number is checked against them and marked when unmatched; the results are shown under each answer.
Explainability	Each recommendation shows its rule, setting, out-of-sample and in-sample result, sample sizes and ranges.
Auditability	"What was sent" shows every request, tool run with its tier, and answer, with exact bodies, downloadable. The server keeps no content.
Retention	Server: one counter per user and day. Provider: `store:false`. Device: log of the last 80 entries, notes cache, rules, paper decisions, all deletable.

AREA	CONTROL
Client hardening	MV3 default CSP, no remote code, no `externally_connectable`. Dashboard CSP `script-src 'self'; object-src 'none'; base-uri 'none'`. Closed shadow root for the in-page card, which ignores synthetic events. Every HTML sink fed by imports, other users or AI replies is escaped; CSV exports neutralise formulas.
Supply chain and CI	Third-party GitHub Actions pinned to commit SHAs; Pages publishes only web-app files; PR previews deploy only after a maintainer label.
Kill switch	Remove a user from the allowlist or unset the key; the rest of the app is unaffected. Provider error text is never returned to the client.

3.5 Security review and known risks

A read-only code review on 17 October 2026 covered database authorisation, edge functions, browser clients, build and hosting, and the mobile app. Each reported issue was re-checked against the code by a second, independent reviewer. It confirmed 3 high, 16 medium and 16 low findings. All three high findings were fixed or partly fixed in code, with the remaining steps listed as owner actions (for example, moving store-publishing secrets to a protected environment). Details are in `docs/SECURITY_ASSESSMENT.md`.

ID	HIGH FINDING	FIX	STATUS
H1	A crowd-overlap function had no minimum group size and returned precise values, from which other users' views could be reconstructed	20-user day and 5-user cell floors; anon access revoked	`fixed`
H2	User feedback text could steer an internal automation agent holding a repository token	Feedback insert locked down; the agent's shell limited; feedback marked untrusted	`partly fixed`
H3	The public site published the whole private repository	Pages deploy limited to web-app files	`fixed`

The main residual risks of the AI layer, from the risk register in `docs/hackathon/SECURITY.md`:

#	RISK	MITIGATION	RESIDUAL
R1	The model states a wrong number or explains a true one wrongly	Numbers checked and marked; results shown	A wrong explanation of a right number is not detected

#	RISK	MITIGATION	RESIDUAL
R2	Personal data typed into a question or a note	Redaction of obvious patterns; preview; consent text	Names in free text cannot be reliably recognised
R3	Re-identification from the AI agent's per-decision records	Aliases, day offsets, request-local ids	A price path matched to public prices can date a record
R4	Free-tier provider terms (content use; no EEA, UK, CH users)	Deny-by-default allowlist; docs state the plan requirement	Depends on the owner configuring it correctly
R7	A rule looks good by luck	Walk-forward with embargo; 99% range; 0 of 216 in null worlds	Overlapping holding periods make ranges somewhat optimistic
R14	Hindsight in the AI agent's backtest	Asked only about moments ≤ 2 days old; saved once; no backfill	Few AI decisions at first
R15	Paper decisions read as advice	Labelled as paper portfolios; acts only on the user's own buys and looks; no order path	A user may still copy a "buy"

3.6 Compliance

- **GDPR principles.** Purpose limitation, data minimisation (section 3.1), consent before any transfer, transparency through "What was sent", storage limitation (nothing kept on the server), and user control (turn off, delete). The transfer to a model provider is declared in the privacy policy and store listing. The backend is hosted in the EU.
- **Financial regulation.** The product describes the user's own behaviour and shows the evidence. It gives no personalised investment advice, no price forecasts and no order execution, which keeps it outside MiFID II investment advice and order handling. Rules are framed as habits; every answer states that it is not investment advice.
- **Model risk.** The model is replaceable through one module. The decisions it influences are limited to choosing read-only tools, wording, and paper decisions; the statistics that drive recommendations contain no model.

4 Tests and reproduction

All suites run in Node 22 with no network, no key and synthetic data. Counts are from a run on 19 October 2026 of build `c711d16`.

SUITE	COVERS	CHECKS	RESULT
`tools/test-ai.js`	AI layer, step validation, privacy filters, every tool, scans of every request	150	pass
`tools/test-trades.js`	Broker import, matching, decisions, the 11 questions	105	pass
`tools/test-indicators.js`	Bias indicators and their edge cases	77	pass
`tools/test-biases.js`	Coach, out-of-sample validation, embargo, null worlds, adherence	39	pass
`tools/test-shadow.js`	Rules and AI agents, look-ahead, what requests carry, holdings	34	pass
`tools/test-pricecache.js`	Price cache, staleness, splits	18	pass
Total		**423**	0 failing

```
node tools/sandbox.js      # the real dashboard on a synthetic investor → http://127.0.0.1:8780/
node tools/demo-run.js     # regenerates the execution evidence
node tools/test-ai.js      # and the other suites above

GEMINI_API_KEY=... node tools/demo-run.js --live   # the same flow on a real model
```

The sandbox runs the `ai-coach` function's own code in the page with a stand-in model, so consent, questions, tools, "What was sent", adopting a rule and checking a buy all work offline.

5 Limits

- The statistics were validated on synthetic investors with planted effects and on worlds with no effect. They have not yet been validated on a population of real investors.

- A rule needs about 24 decisions with a known outcome before it is tested, and 8 flagged out of sample before it can be called supported. Light traders will often see "not enough data".
- The model request and reply shapes follow the public documentation and are tested against a fake model; `tools/ai-smoke.js` checks them against the live API with a key.
- The free model tier may be used only on synthetic or demo data and not for users in the EEA, UK or Switzerland. Real users need a paid plan.
- Group-size floors raise the cost of re-identification but do not stop an attacker with many accounts. Noise or rate limits would be the next step.

Source documents in the repository: `ARCHITECTURE.md` (sections 2 to 6, 8b, 8c), `docs/AI.md`, `docs/hackathon/TECHNICAL.md`, `docs/hackathon/SECURITY.md`, `docs/SECURITY_ASSESSMENT.md`, `docs/hackathon/evidence/demo-run.md`. Nothing in this document or in the product is investment advice.